Verifiability Voters Can Use: Second-Device Ballot Checking and Witnessed Bulletin Boards in D-Voting

Pranav Kumar Singh[1], **Dr. Kavi Priya D**[2]

[1]Department of Computing Technologies, Faculty of Engineering and Technology,
SRM Institute of Science and Technology (SRMIST), Kattankulathur, Chengalpattu, Tamil Nadu, India
M.Tech – Computer Science and Engineering (Full Stack), Roll No. RA2512055010005

[2]Department of Computing Technologies, Faculty of Engineering and Technology,
SRM Institute of Science and Technology (SRMIST), Kattankulathur, Chengalpattu, Tamil Nadu, India
Corresponding author: ps9567@srmist.edu.in

Abstract

End-to-end verifiable (E2E-V) voting promises that voters need not trust the machines that record and count their votes. In practice two trust assumptions survive. First, the cast-as-intended check (the Benaloh challenge) is usually performed on the same device that prepared the ballot, so a compromised voting app can simply report that its own test passed. Second, every check a voter runs reads the public bulletin board through the server being checked, so a dishonest server can show one voter a private board (a split view) on which every check succeeds. We present D-Voting, an open-source E2E-V system that removes both assumptions with mechanisms an ordinary voter can operate. (i) A **second-device check**: before the voter decides to cast or test, the voting screen shows a QR code carrying the ballot's commitment; the voter's phone then reads the sealed ballot from the public board and re-runs the audit itself, showing which candidate the ballot really contains, named from the sealed election record so that even a falsified candidate list is exposed. (ii) A **witnessed bulletin board**: independent observers keep their own replicas, refuse to countersign any rewrite, fork or rollback, and sign checkpoints that voters' devices fetch directly; a device accepts a receipt only when a quorum of witnesses' heads is reachable from the voter's block through hash links it recomputes, which detects split views and, for auditors, omission. (iii) A **voter interface** built so that each guarantee is visible as it happens: an EVM-style ballot with party symbols and NOTA, English and Hindi, a live "who knows what" panel, a 12-digit check code and a receipt timeline. The system composes RSA blind-signature credentials (RFC 9474), exponential ElGamal with disjunctive Chaum–Pedersen proofs, Pedersen DKG with threshold decryption, and a permissioned Ed25519 ledger with RFC 6962 Merkle proofs. On a laptop (i5-1135G7, one core) a 4-candidate ballot is prepared in 149 ms at 2048-bit and 473 ms at 3072-bit; the phone confirms a cast ballot in 2.8 ms and its witness evidence in 3.9 ms, and re-runs a tested ballot's audit in 203 ms. An independent audit costs 192 ms per ballot. The implementation passes 578 automated tests, including end-to-end split-view and manipulated-client attacks. Building the verification features also exposed four integrity bugs in sealing that 530 existing tests had missed. We describe a counterbalanced user study (N = 30) measuring whether participants detect a manipulated client with each check; its results will be reported in the final version.

Keywords: end-to-end verifiability; cast-as-intended; Benaloh challenge; second-device verification; bulletin board; split-view attack; witness cosigning; usable security; electronic voting.

1 Introduction

India's Election Commission proposed a multi-constituency remote voting machine for the country's domestic migrant workers in 2022–23, citing internal migration as a prominent reason for the 67.4% turnout of 2019. The proposal met unanimous opposition from political parties on grounds of trust, not engineering [1]. The episode illustrates the central problem of remote and electronic voting: a system must not merely be secure, it must let the people it serves check that it is.

End-to-end verifiable voting answers this in principle. Ballots are encrypted on the voter's device; voters check that their ballot was cast as intended, recorded as cast, and counted as recorded; and anyone can recount from public evidence [2, 3]. Deployed and research systems such as Helios [2], Belenios [4], ElectionGuard [5] and the Estonian IVXV [6, 7] realise much of this. Two residual trust assumptions nevertheless recur, and both are visible in the original D-Voting prototype on which this work builds.

The checking device is the voting device. The Benaloh challenge [3] lets a voter test a prepared ballot: the client must reveal how it encrypted the choice, and a client that cheated is caught. The test, however, must be checked by something the client does not control. Run on the voting device itself, a malicious app reports "test passed". Our own earlier threat model listed this as the single largest residual risk.

The board is read through the server. Every voter check, whether an inclusion proof, validator signatures or a recount, reads the bulletin board through the ballot box. Our earlier verifier even took the validator keys from the ballot box. A dishonest box can therefore build a private chain for one voter, sign it with keys it invented, serve those keys alongside, and every check on that voter's device passes. Hirschi, Schmid and Basin call the bulletin board the "Achilles heel" of e-voting for this reason [8].

Usability compounds both. Studies of E2E-V systems found that many voters failed to cast, some believed they had cast when they had not [9], and terms such as "audit" and "ballot fingerprint" were not understood [10].

Contributions.

1. **A second-device cast-as-intended check** that a voter operates with a phone camera (§5). The phone reads only public data, re-runs the audit of a tested ballot, confirms a cast ballot is the one committed to, and names the encoded candidate from the sealed election record, which also exposes a client that displayed a falsified candidate list.
2. **A witnessed bulletin board with device-side split-view detection** (§6). Independent witnesses countersign the board's head; devices fetch checkpoints directly and verify hash-chain reachability from the voter's block. A 12-digit election fingerprint anchors the scheme, and auditors additionally detect omitted blocks.
3. **A voter interface derived from the system's principles** (§7): an EVM-style ballot with symbols and NOTA, English and Hindi, a live separation-of-knowledge panel, human-comparable codes and a receipt timeline.
4. **An open implementation and evaluation** (§8–9): 578 automated tests including end-to-end attacks, a measured performance evaluation, and four concurrency bugs in ballot sealing found and fixed in the process.
5. **A user-study protocol** (§10) for measuring manipulation detection with same-device and second-device checks.

2 Background and related work

E2E-V systems. Helios [2] brought web-based open-audit voting with a Benaloh challenge. Belenios [4] provides privacy and verifiability against a compromised voting server. ElectionGuard [5] is a toolkit, used in public elections in several US states, that adds verifiability to existing voting systems. Estonia's IVXV [6, 7] lets voters check their ballot with a verification app on a second device by scanning a QR code, and has added ballot-validity proofs to the live system [7]. Swiss research continues to remove trusted components from the national protocol [11]. Haines and Rose show that carefully designed and audited deployed systems still harbour "vestigial" weaknesses [12], which argues for designs whose remaining assumptions are few and stated.

Cast-as-intended and its usability. The Benaloh challenge [3] forces a client to commit before learning whether a ballot will be cast or audited. Usability work found Benaloh variants and tracking codes easy to use when nothing was wrong, but detection of manipulation varied widely across designs, from none to most participants [10]. Second-device verification is the approach IVXV takes [6]; our contribution is a second-device check over public data that also covers the cast ballot, names the candidate from the sealed record, and is integrated with witnessed board consistency.

Bulletin boards and split views. A board that shows different parties different contents breaks verifiability [8]; Hirschi et al. define final agreement and give a protocol with machine-checked proofs [8]. Outside voting, Certificate Transparency [13] and witness cosigning [14] make logs consistent by having independent parties countersign a log's head. We adapt the witness idea to an election board, with the verification done on the voter's own device, and connect it to election-specific checks.

Blockchain voting. Park, Specter, Narula and Rivest argue that blockchains do not solve internet voting's core problems and can worsen them [15]. D-Voting follows that argument: its permissioned ledger provides tamper evidence, while secrecy and integrity come from the cryptographic protocol.

3 Goals and threat model

The goals are eligibility, ballot secrecy, cast-as-intended, recorded-as-cast, counted-as-recorded, tamper evidence, and immutability of the election's rules once open. Adversaries are outside attackers, malicious voters, a **compromised voting client** (A3), a dishonest commission or ballot-box operator (A4), a dishonest registration authority, up to f of $n > 3f$ faulty validators, up to $k - 1$ colluding trustees, a coercer, and a **forger of the record** (A9).

This paper targets A3 and the combination of A4 and A9: a ballot box that serves a targeted voter a board no one else sees. We assume at least the witness quorum of witnesses is honest and reachable; that the election fingerprint is published through channels the ballot box does not control; and that the voter scans the QR code with a device the voting client does not control. Coercion resistance, timing side channels and identity proofing are out of scope and listed in §11.

4 System overview

Five kinds of party each hold one secret and learn one thing.

Party	Holds	Learns	Never learns
Voter's device	choice, credential, encryption randomness	—	—
Registration Authority	issuer key	which roll entry registered	the credential it signed
Ballot box	nothing secret	encrypted ballots	who cast them
Validators ($n = 4$, quorum 3)	one Ed25519 key each	sealed blocks	any choice
Trustees (3 of 5)	one key share each	only the totals, jointly	any single ballot
Witnesses (2 of 3)	one Ed25519 key each	the board's head	any choice

A voter proves eligibility with a polling card and receives an RSA blind-signature credential (RFC 9474 [16]) that the authority cannot link to them. The browser encrypts the choice with exponential ElGamal over an RFC 3526 group [17], with disjunctive Chaum–Pedersen proofs [18, 19] made non-interactive with the strong Fiat–Shamir transform [20]. The ballot box checks the credential and proofs and seals ballots into a hash-chained ledger whose blocks carry RFC 6962 Merkle roots [13] and a validator quorum of Ed25519 signatures [21]. Opening the poll seals the whole election definition into block 0: candidates and their presentation, selection limits, schedule, issuer key, election key (from Pedersen DKG [22]), trustee roster, validator set, roll commitment and witness roster. After closing, trustees verify the chain, recompute the encrypted totals, and contribute proved partial decryptions of the totals only. The result is sealed with every proof, so anyone can recount.

5 Second-device cast-as-intended check

Commitment. When the voter reviews a prepared ballot, the client has already computed the commitment c = SHA-256("dvoting/ballot-commitment/v1" || id || α_1 || β_1 || … || α_m || β_m) over the ballot id and every ciphertext. The review screen shows forty bits of c as a 12-digit **check code** in three groups, which is readable in any script and comparable between devices, together with a QR code encoding `/check#e={election}&b={ballot id}&c={commitment}`. The values travel in the URL fragment, which browsers never send to a server. The QR code appears **before** the Cast and Test buttons, so the phone holds c before the client learns which way the voter will go.

Tested ballot. If the voter chooses "Test this ballot instead", the client must publish the ballot with its randomness; the ballot box seals it as a spoiled ballot. The phone fetches the sealed entry's inclusion receipt from the board, decodes the leaf, checks that its kind and id match the link, re-parses the ballot with the same range checks as the server, verifies its proofs, and re-encrypts the revealed selection with the revealed randomness, requiring a byte-exact match with the ciphertexts under commitment c. It then shows "This tested ballot contained:" with the candidate's name and party taken from the **sealed** record and the symbol drawn locally, and asks "Is this exactly what you selected?".

Cast ballot. The phone waits until the ballot id is sealed as cast, verifies the inclusion proof and validator quorum, recomputes the commitment of the sealed ciphertexts and requires it to equal c. If nothing is sealed within 90

seconds of the voter's decision, it warns that the ballot was not recorded as shown.

What it catches. Table 1 lists each way a client can cheat; each row is an automated test that drives the real ballot box and the phone-side code.

Table 1. Client misbehaviour and the second-device check.

Client behaviour	Phone's result
Encrypts another candidate, reveals honestly when tested	Shows the true candidate; the voter rejects it
Encrypts another candidate, lies about it when tested	Audit fails: randomness does not reproduce the ciphertexts
Shows ballot X's code, reveals a different honest ballot Y when tested	Fails: Y does not match commitment c
Shows ballot X's code, casts a different ballot Y	Fails (commitments differ), or X never seals and the phone warns
Displays a falsified candidate list	Phone names the encoded candidate from the sealed record

Because the client commits before knowing the voter's decision, it cannot cheat only on cast ballots. A client that alters a fraction p of ballots survives one test with probability $1 - p$. The check is probabilistic per voter but compounding across an electorate, as for any Benaloh challenge.

6 Witnessed bulletin board

Roster. The commission configures witnesses, each an id, an Ed25519 key and a URL, and a quorum k. The roster is sealed in block 0.

Witness rules. A witness keeps a durable replica and syncs every few seconds. It accepts block h only if it links to the witness's head, carries a validator quorum, and passes the same validation as every replica. If the board shrinks, presents a different head at a height already seen, or offers a block that does not extend the witness's history, the witness raises a **sticky alarm** and stops signing; deciding that a fork was benign is left to a human. Otherwise it signs a checkpoint over the canonical encoding

```
encode("dvoting/checkpoint/v1", witnessId, electionId,
       u32 height, headHash, genesisHash, witnessedAt).
```

The witness id is inside the signed bytes, so a signature cannot be re-attributed. The replica is re-verified on restart, so restarting a witness does not make it forget.

Device verification. For a receipt in block b (Algorithm 1), the device takes the roster from block 0's configuration entry, never from the ballot box's self-description; fetches each witness's checkpoint from the witness's own URL; fetches only block headers from b upward; and recomputes the hash chain.

Algorithm 1. Accept a receipt as witnessed.

```
G ← hash(header of block 0);  R ← roster sealed in block 0
H[b] ← hash(receipt header);  for h > b: H[h] ← hash(header h) if header h links to H[h−1]
vouched ← {}
for each witness w in R:
    cp ← checkpoint fetched from w.url
    if sig valid under w.key and cp.election = e and cp.genesis = G
       and cp.height > b and H[cp.height - 1] = cp.headHash:  add w to vouched
accept iff |vouched| ≥ R.quorum   (pending if not yet covered; any alarm is shown)
```

A head the links do not reach means that witness saw a **different board**. The device also displays twelve digits of G as the **election fingerprint**.

Auditors. The standalone verifier, given checkpoints, requires the downloaded board to contain every block each witness saw, exactly as seen. A board shorter than a checkpoint fails, which detects omission, for example a hidden result or late ballots. Each witness also serves its replica in the ballot box's format, so auditors can download the board from a witness.

Security argument. Suppose the ballot box shows a targeted voter a private board. If it reuses the real block 0, its private blocks above the last witnessed head do not link to any honest witness's head, so the honest witnesses are rejected and the receipt fails, provided at least k witnesses are honest. If it substitutes its own block 0, every honest witness's genesis hash differs and is rejected. The only remaining attack is to fabricate the entire election, including

a roster of witnesses it controls. That produces a different block 0 and therefore a different fingerprint, which only comparison with the published value reveals; no protocol can do better, because the forger controls everything the device can reach. Honest witnesses never sign two inconsistent heads, so a forked history cannot be shown to everyone either.

7 Voter interface

Each design decision answers a finding (Table 2). The ballot is the first thing on screen at every width; the earlier interface showed only a marketing banner on a phone's first screen.

Table 2. Findings and design responses.

Finding	Response
Voters failed to cast, or believed they had cast when they had not [9]	Task-first layout; explicit end states; a receipt timeline that turns green only when the public record confirms it
"Audit", "fingerprint" not understood [10]	Plain language: the ballot is "locked"; "Test this ballot instead", with one sentence on why it is used up
A 43-character fingerprint cannot be compared	12-digit check code; QR code for the phone
Voters recognise candidates by symbol; NOTA is mandatory on Indian ballots [23]	EVM-style rows: serial, name, party, symbol, large target; optional NOTA; presentation sealed in block 0
Many voters read Hindi first	Every voter-facing string in English and Hindi, switched in place without a reload, which would destroy the in-memory credential
Separation of knowledge is the central claim	A "Who knows what, right now" panel, updated at each step, per party
Small type excludes older voters	16 px base, nothing under 13 px, 50 px targets, real form controls (WCAG 2.2)

The symbols are a fixed, neutral set drawn by the browser; the record names a symbol rather than carrying an image, so no markup enters the chain, and none is a real party's registered symbol.

8 Implementation

D-Voting is a TypeScript monorepo of eight workspaces (crypto, ledger, verifier, registration authority, ballot box, validator, trustee, witness) running directly on Node's type stripping with no build step, except for two browser bundles (the crypto library and a QR encoder) whose SHA-256 digests are published and reproduce from source. The cryptographic package uses WebCrypto and native BigInt with no third-party cryptographic dependency. Pages run under a strict Content-Security-Policy with no inline script or style. The test suite has **578 tests**, including end-to-end attacks: lying, swapping and substituting clients (13), split views, rewritten history, impostor witnesses and alarm propagation (16), and omission detection in the verifier (6).

Bugs found by building verification. Testing the second-device check under load exposed four integrity bugs in ballot sealing that the existing 530 tests had missed. (1) A ballot accepted while a block was replicating was cleared without being sealed: accepted, given a tracking code, and lost. (2) A ballot accepted during closing could be sealed after the close record. (3) Two identical ballots submitted together could both be accepted, producing a block the ledger must refuse and jamming every ballot behind it. (4) Blocks sealed only when a count threshold was reached, so a voter's receipt could wait indefinitely. The fixes serialise block writes, remove only sealed entries, close the poll before sealing the close record, add a synchronous final acceptance check, and seal on a timer. Each has a regression test written as an invariant over all interleavings. We note this because verification features are usually argued to protect against attackers; here, building one also protected against the system's own bugs.

9 Evaluation

Setup. All measurements use the real code paths in one Node 24 process on one core of an Intel i5-1135G7 laptop with 8 GB RAM, without network latency, and are reproducible with `node scripts/bench/evaluate.ts` (commit `28aa4df`). The system run uses 4 validators, 3 witnesses and 40 ballots at 2048-bit; device micro-benchmarks also cover 3072-bit, the production default.

Table 3. Voter-device and phone costs (median, ms).

Operation	2048-bit, 4 cand.	2048-bit, 8 cand.	3072-bit, 4 cand.
Prepare ballot (encrypt, prove, commit)	149	301	473
Verify ballot proofs (per observer, per ballot)	240	440	711
Second-device audit of a tested ballot	254	473	763

Table 4. System and phone costs, 2048-bit, 3 candidates (median, ms).

Operation	Median	p95
Ballot-box admission (credential, proofs, duplicates)	185	199
Seal one block, 4 validators (consensus only)†	5.3	7.2
Witness: fetch, validate, countersign one block	2.1	3.4
Phone: confirm a cast ballot	2.8	3.1
Phone: re-run a tested ballot's audit	203	204
Phone: witness confirmation, 3 checkpoints	3.9	5.2
Independent audit, per ballot (recount + witnesses)	192	—

† Deployed validators additionally re-verify each ballot (one "verify ballot proofs" each, in parallel on their own machines).

Witness check versus distance to the head. The device's cost grows with the number of headers between the voter's block and the witnessed head: 1.6 ms at 9 blocks, 4.6 ms at 99, 39 ms at 999 and 190 ms at 4,999 — linear and small, because headers are 149 bytes and only hashed.

Sizes. A ballot occupies about 12.4 kB on the board; a device fetches a 17.4 kB inclusion receipt and 323-byte checkpoints; the QR link is 118 characters, well within a QR code readable from a laptop screen.

Scaling. Auditing is linear in ballots: from the measured 192 ms per ballot, a full independent audit of 100,000 ballots takes about 5.3 hours on one core, or about 40 minutes if verification is parallelised across 8 cores (projected; the current CLI uses one core). The exported board is about 2 GB per 100,000 ballots with one ballot per block. Ballot-box admission at 185 ms per ballot bounds a single core to about 19,000 ballots per hour; admission is embarrassingly parallel across processes behind the sealing queue.

Discussion. The voter-facing costs that matter for usability are well under a second at 2048-bit and under one second at 3072-bit on a laptop core; phones are slower, and measuring mid-range Android devices is future work. The phone's cast-ballot and witness checks are dominated by network round trips rather than computation.

10 User-study design

To test whether voters actually catch a manipulated client, we designed a within-subjects study (protocol, scripts and materials in `docs/user-study/`). Thirty participants each vote four times for candidates named on cards (so no political preference is involved), under two counterbalanced conditions: (A) the same-device test, and (B) the second-device phone check. In each condition one trial is manipulated: a study build, enabled only with an explicit flag and refused in production, makes the client encrypt the next candidate while showing the participant's choice and reporting that its own test passed. We verified this behaviour end to end: the voting screen showed "Rohit Nair" and "Test passed", while the phone showed that the ballot contained "Meera Iyer". Measures are detection before casting (McNemar test), SUS per condition [24] (Wilcoxon signed-rank, non-inferiority margin 10 points), task time, phone-check completion and a five-item comprehension score. Participants are debriefed and may withdraw their data. The study is pending institutional ethics approval; **its results will be reported in the final version of this paper.**

11 Security analysis and limitations

The mechanisms rest on stated assumptions: fewer than k colluding trustees; fewer than a third faulty validators; at least the quorum of witnesses honest; a published election fingerprint; a checking device the voting client does not control; and that the browser code is the published code. The phone check and the witness check both still trust the code served to the phone; publishing the check page from an independent origin would remove that. Browsers expose WebCrypto only on secure origins, so the phone page requires HTTPS and says so when it is missing.

Not addressed: coercion resistance in the JCJ sense (re-voting is mitigation only); vote buying; timing side channels in BigInt arithmetic; identity proofing; hardware key custody; post-quantum security; and the correctness of the electoral roll, which the roll commitment makes tamper-evident but cannot make correct. The manipulation detection rate is a property of voters and must be measured, which is the purpose of §10.

12 Conclusion

The trust that E2E-V removes in theory often returns in practice through the voting device and through the bulletin board. D-Voting shows that both can be removed with mechanisms an ordinary voter operates: a phone that checks a ballot from public data, and witnesses whose checkpoints the voter's device reads directly. The implementation, tests, benchmarks and study materials are public, and the remaining assumptions are listed rather than hidden.

References

[1] Election Commission of India, "Remote Voting: Concept Note," Dec. 2022; and contemporaneous reporting of political parties' objections, e.g. *Outlook India*, Jan. 2023.

[2] B. Adida, "Helios: Web-based Open-Audit Voting," *USENIX Security Symposium*, 2008.

[3] J. Benaloh, "Simple Verifiable Elections," *USENIX/ACCURATE Electronic Voting Technology Workshop (EVT)*, 2006.

[4] V. Cortier, P. Gaudry and S. Glondu, "Belenios: A Simple Private and Verifiable Electronic Voting System," in *Foundations of Security, Protocols, and Equational Reasoning*, LNCS 11565, Springer, 2019.

[5] J. Benaloh et al., "ElectionGuard: a Cryptographic Toolkit to Enable Verifiable Elections," *USENIX Security Symposium*, 2024.

[6] S. Heiberg and J. Willemson, "Verifiable Internet Voting in Estonia," *International Conference on Electronic Voting (EVOTE)*, 2014.

[7] T. Kraavi and J. Willemson, "Proving vote correctness in the IVXV internet voting system," *Scientific Reports* 15, 31793, 2025.

[8] L. Hirschi, L. Schmid and D. Basin, "Fixing the Achilles Heel of E-Voting: The Bulletin Board," *IEEE Computer Security Foundations Symposium (CSF)*, 2021.

[9] C. Z. Acemyan, P. Kortum, M. D. Byrne and D. S. Wallach, "Usability of Voter Verifiable, End-to-end Voting Systems: Baseline Data for Helios, Prêt à Voter, and Scantegrity II," *USENIX Journal of Election Technology and Systems (EVT/WOTE)*, 2014.

[10] K. Marky, O. Kulyk, K. Renaud and M. Volkamer, "What Did I Really Vote For? On the Usability of Verifiable E-Voting Schemes," *ACM CHI Conference on Human Factors in Computing Systems*, 2018.

[11] V. Cortier, A. Debant, A. Esseiva, P. Gaudry, A. Høgåsen and C. Spadafora, "A Practical and Fully Distributed E-Voting Protocol for the Swiss Context," IACR ePrint 2025/1625.

[12] T. Haines and J. Rose, "Vestigial Vulnerabilities in Deployed Verifiable E-Voting Systems," IACR ePrint 2025/2033.

[13] B. Laurie, A. Langley and E. Kasper, "Certificate Transparency," RFC 6962, 2013.

[14] E. Syta et al., "Keeping Authorities 'Honest or Bust' with Decentralized Witness Cosigning," *IEEE Symposium on Security and Privacy*, 2016.

[15] S. Park, M. Specter, N. Narula and R. L. Rivest, "Going from bad to worse: from Internet voting to blockchain voting," *Journal of Cybersecurity* 7(1), 2021.

[16] F. Denis, K. Jacobs and C. A. Wood, "RSA Blind Signatures," RFC 9474, 2023.

[17] T. Kivinen and M. Kojo, "More Modular Exponential (MODP) Diffie-Hellman groups for Internet Key Exchange (IKE)," RFC 3526, 2003.

[18] D. Chaum and T. P. Pedersen, "Wallet Databases with Observers," *CRYPTO*, 1992.

[19] R. Cramer, I. Damgård and B. Schoenmakers, "Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols," *CRYPTO*, 1994.

[20] D. Bernhard, O. Pereira and B. Warinschi, "How Not to Prove Yourself: Pitfalls of the Fiat-Shamir Heuristic and Applications to Helios," *ASIACRYPT*, 2012.

[21] S. Josefsson and I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017.

[22] T. P. Pedersen, "A Threshold Cryptosystem without a Trusted Party," *EUROCRYPT*, 1991.

[23] Supreme Court of India, *People's Union for Civil Liberties v. Union of India*, (2013) 10 SCC 1.

[24] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, Taylor & Francis, 1996.

Appendix A: Reproducibility

```
npm install && npm run build:web
npm test                              # 578 tests
node scripts/bench/evaluate.ts        # regenerates papers/evaluation/results.md
npm run dev                           # full system: 4 validators, 5 trustees, 3 witnesses
STUDY_MODE=on npm run dev             # study build (never in production)
```

Design notes: `docs/witnesses.md`, `docs/cast-as-intended.md`, `docs/frontend-design.md`, `docs/threat-model.md`. Figures in `papers/figures/`.